1

“Gang of Four” (GoF) Patterns

The Fundamental OO Design Patterns

Includes content adapted from *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, Addison-Wesley && *Java Design Patterns, 2022, Apress*



Previously on Design Patterns ...

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioral patterns

			Purpose	
		Creational	**Structural**	**Behavioral**
Scope	**Class**	1. Factory Method	1.1. Adapter(class)	1. Interpreter 2. Template Method
	Object	2. Singleton 3. Prototype 4. Builder 5. Abstract Factory	1.2. Adapter(object) 2. Proxy 3. Flyweight 4. Composite 5. Bridge 6. Facade 7. Decorator	3. Observer 4. Strategy 5. Command 6. Iterator 7. Memento 8. State 9. Mediator 10. Visitor 11. Chain of Responsibility



Previously on Design Patterns ...

Today's topic

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioural patterns

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	1. Factory Method	1.1. Adapter(class)	1. Interpreter 2. Template Method
	Object	2. Singleton 3. Prototype 4. Builder 5. Abstract Factory	1.2. Adapter(object) 2. Proxy 3. Flyweight 4. Composite 5. Bridge 6. Facade 7. Decorator	3. Observer 4. Strategy 5. Command 6. Iterator 7. Memento 8. State 9. Mediator 10. Visitor 11. Chain of Responsibility

1

Behavioral Patterns

How to distribute behavior between classes and objects.



Behavioral Patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.
- They describe not just patterns of objects or classes but also the patterns of communication between them.



Behavioral Patterns

1. **Template Method**
2. Observer
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter

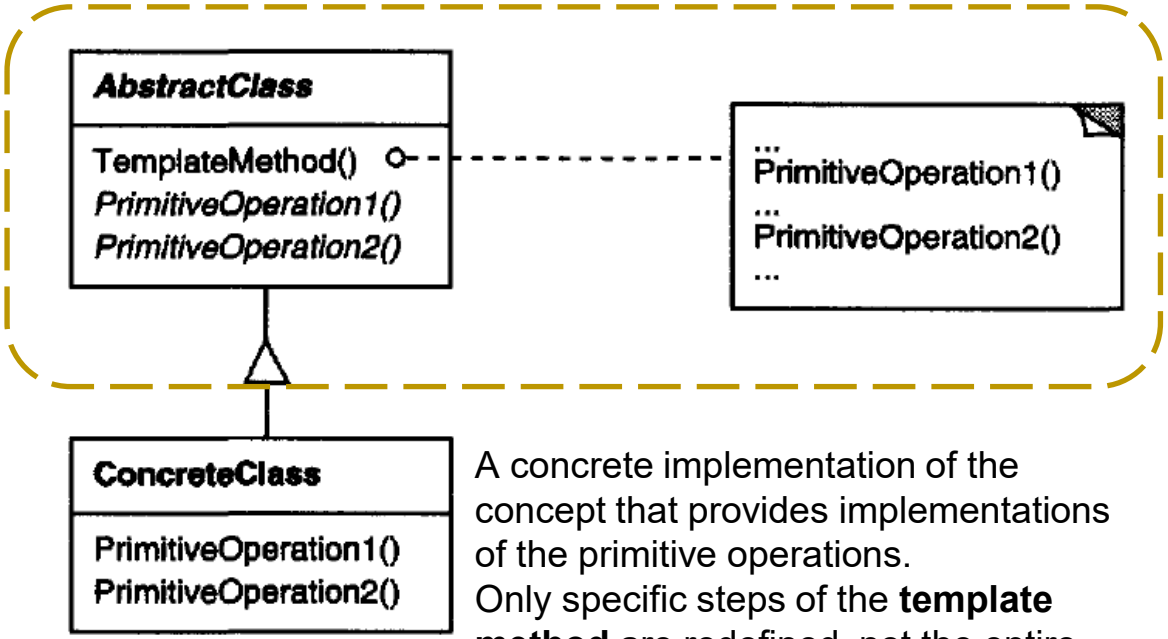


Template Method

- It defines the skeleton of an algorithm in an operation, deferring some steps to subclasses.
- The **Template Method** pattern lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Java API example: Event processing methods (e.g. `onKeyPressed()`) are examples of a **Template Method** pattern.

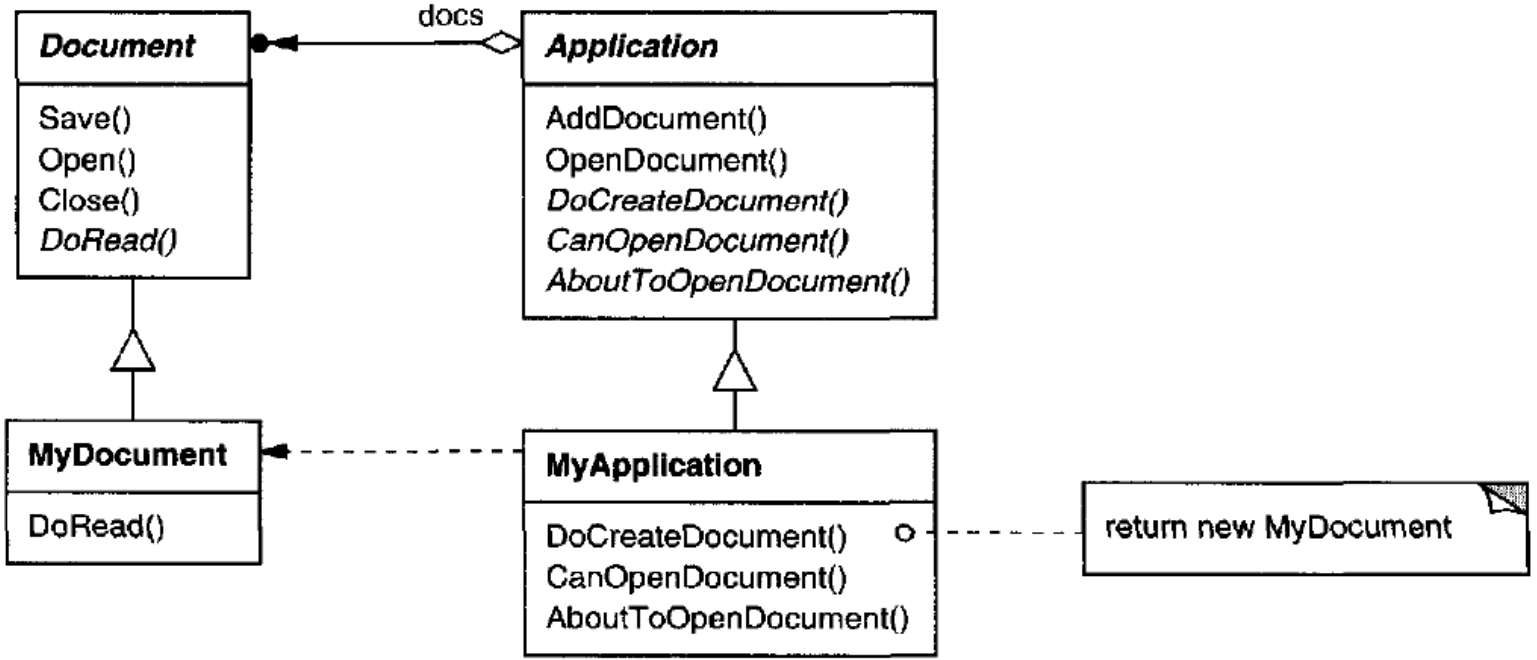
Template Method



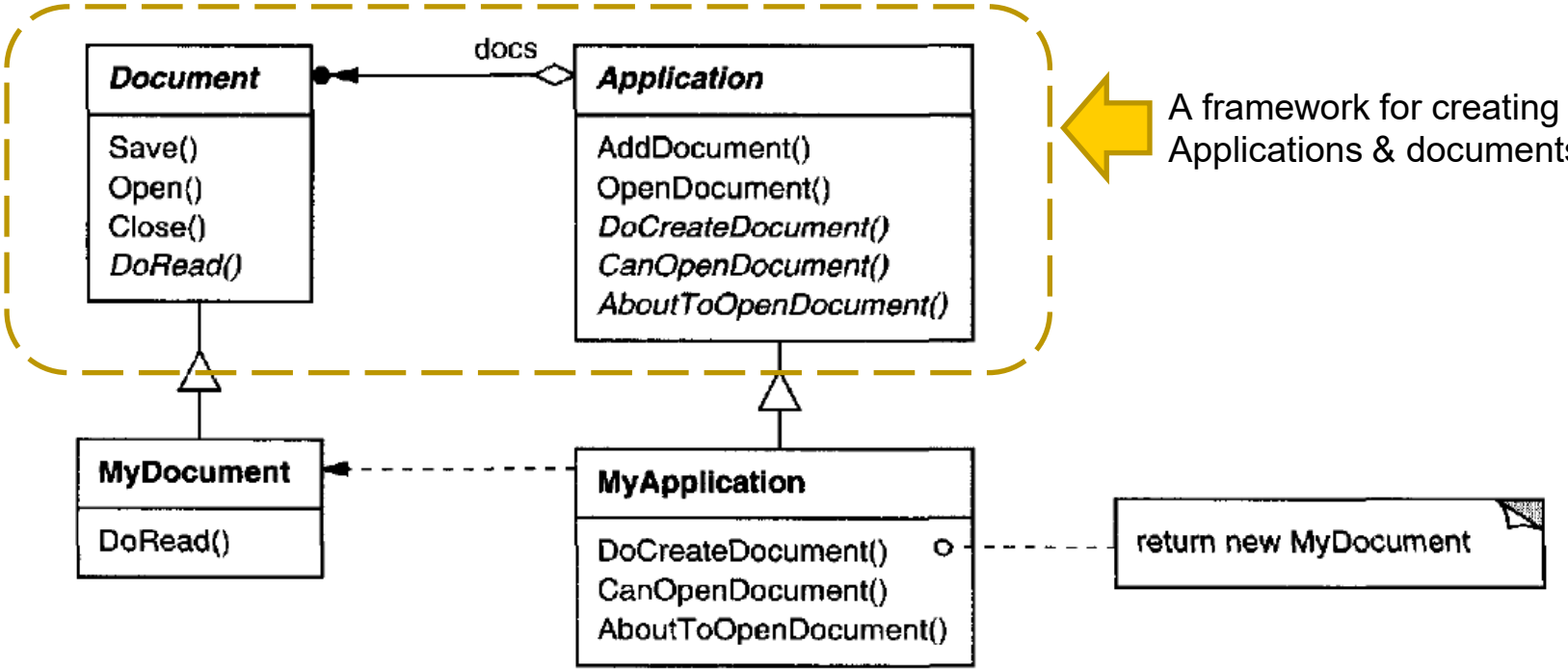
A concrete implementation of the concept that provides implementations of the primitive operations.
Only specific steps of the **template method** are redefined, not the entire method/algorithm.

An abstract representation of a concept that provides an implementation of an algorithm based on abstract primitive operations.
It is intended to be provided as a generic framework or library.
AKA the **template class**.
Some primitive operations might be provided with default implementations. These are called **hooks**.

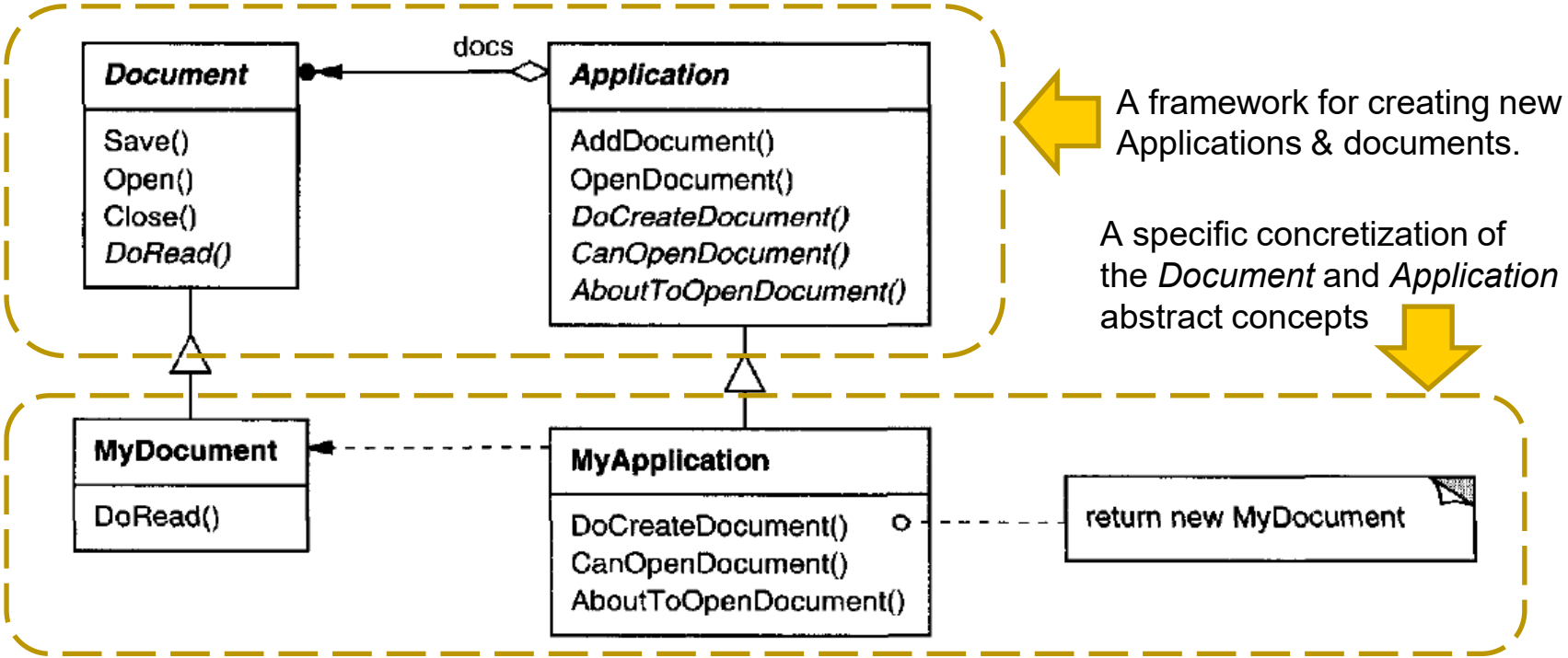
Template Method



Template Method

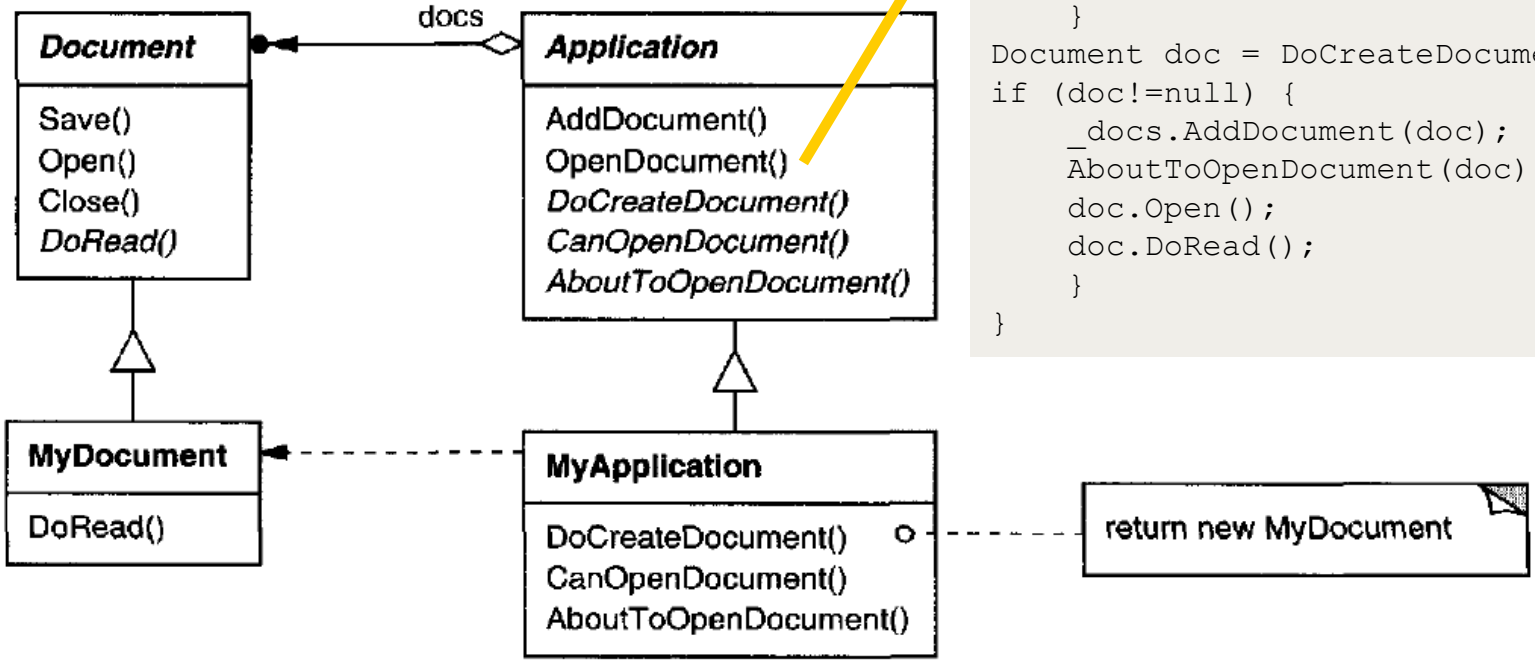


Template Method



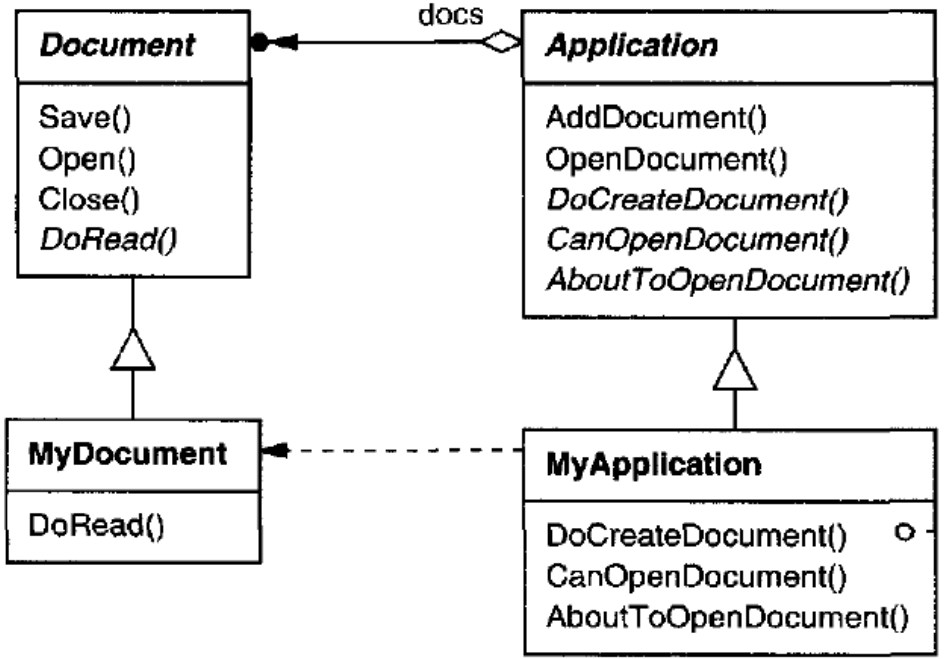
Ten

`OpenDocument` is a **template method**. It specifies the generic steps (algorithm) required for opening a document, regardless of its specific type.




```
void OpenDocument(String name)
if (!CanOpenDocument(name)) {
    // cannot handle this document
    return;
    }
Document doc = DoCreateDocument();
if (doc!=null) {
    _docs.AddDocument(doc);
    AboutToOpenDocument(doc);
    doc.Open();
    doc.DoRead();
    }
}
```

Template Method



`Application` subclasses provide the implementation of methods that check if a file can be opened, and also creating the document.
`Document` subclasses define the method that actually reads the document.
The `AboutToOpenDocument()` method is used to notify concrete implementations that a document is about to be opened, just in case they care.



Template Method

- This pattern is used when creating code intended to be reused.
- Very useful for class libraries, they factor out and allow reuse of common behavior, while still allowing for customization.



Behavioral Patterns

1. Template Method
2. **Observer**
3. Chain of Responsibility
4. Iterator
5. Command
6. Memento
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter

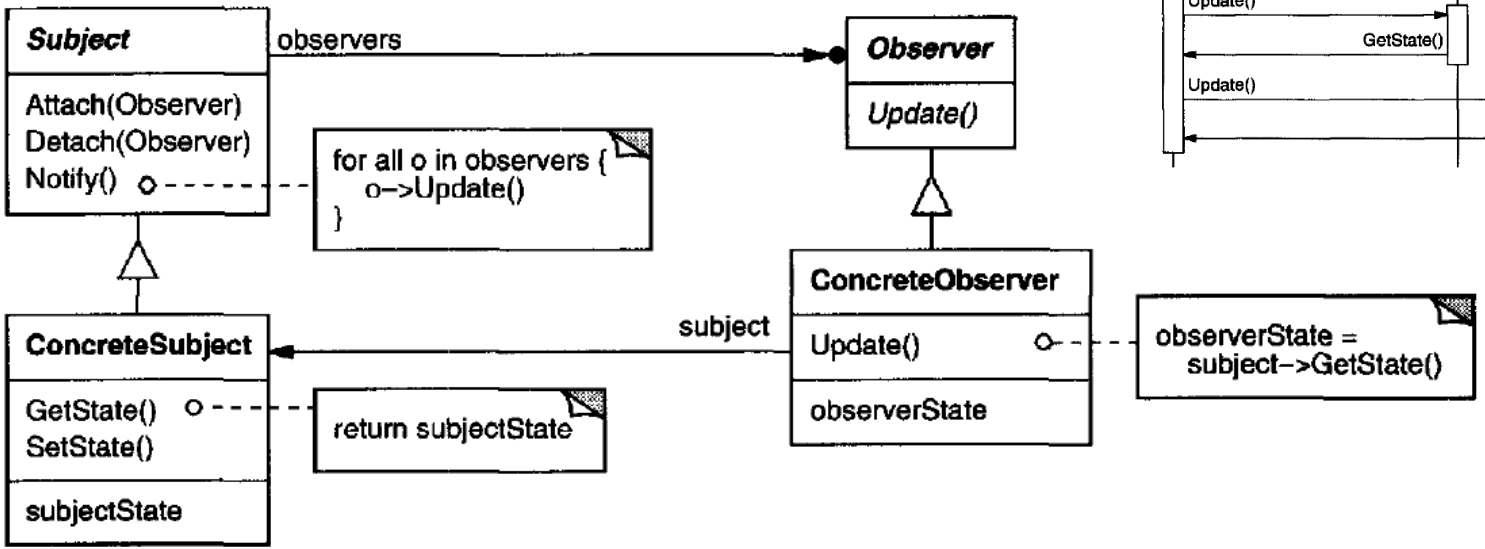


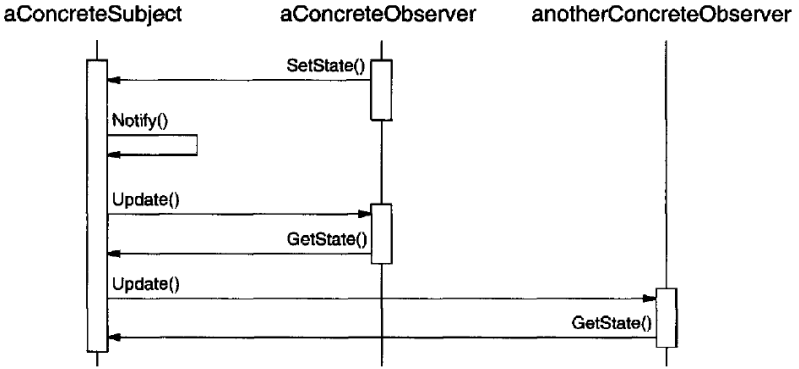
Observer

- It defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- Allows consistency to be maintained while avoiding the need for high coupling.

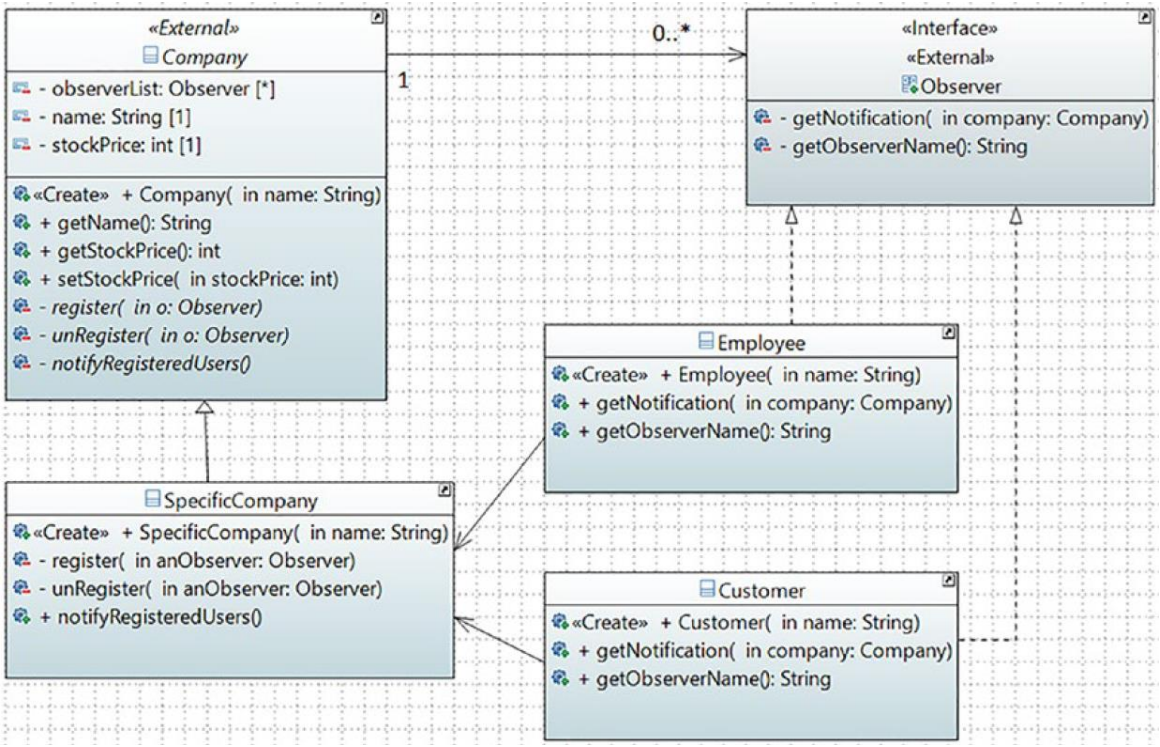
Java API example: The `Observer` class and `Observable` interface (up to Java 8), the `java.beans` listener package (Java 9+).

Observer


Subject
observers
Attach(Observer)
Detach(Observer)
Notify()
for all o in observers {
o->Update()
}
Observer
Update()
ConcreteSubject
GetState()
SetState()
subjectState
return subjectState
subject
ConcreteObserver
Update()
observerState
observerState =
subject->GetState()


aConcreteSubject
aConcreteObserver
anotherConcreteObserver
SetState()
Notify()
Update()
GetState()
Update()
GetState()

Observer


«External»
Company
- observerList: Observer [*]
- name: String [1]
- stockPrice: int [1]
«Create» + Company(in name: String)
+ getName(): String
+ getStockPrice(): int
+ setStockPrice(in stockPrice: int)
- register(in o: Observer)
- unRegister(in o: Observer)
- notifyRegisteredUsers()
0..*
1
«Interface»
«External»
Observer
- getNotification(in company: Company)
- getObserverName(): String
Employee
«Create» + Employee(in name: String)
+ getNotification(in company: Company)
+ getObserverName(): String
SpecificCompany
«Create» + SpecificCompany(in name: String)
- register(in anObserver: Observer)
- unRegister(in anObserver: Observer)
+ notifyRegisteredUsers()
Customer
«Create» + Customer(in name: String)
+ getNotification(in company: Company)
+ getObserverName(): String



Observer

Use when...

- An abstraction has two aspects, one dependent on the other.
  - Encapsulating these aspects in separate objects lets you vary and reuse them independently.
- A change to one object requires changing others, and you don't know how many objects need to be changed.
- An object should be able to notify other objects without making assumptions about who these objects are.
  - In other words, you don't want these objects tightly coupled.



Behavioral Patterns

1. Template Method
2. Observer
3. **Chain of Responsibility**
4. Iterator
5. Command
6. Memento
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter



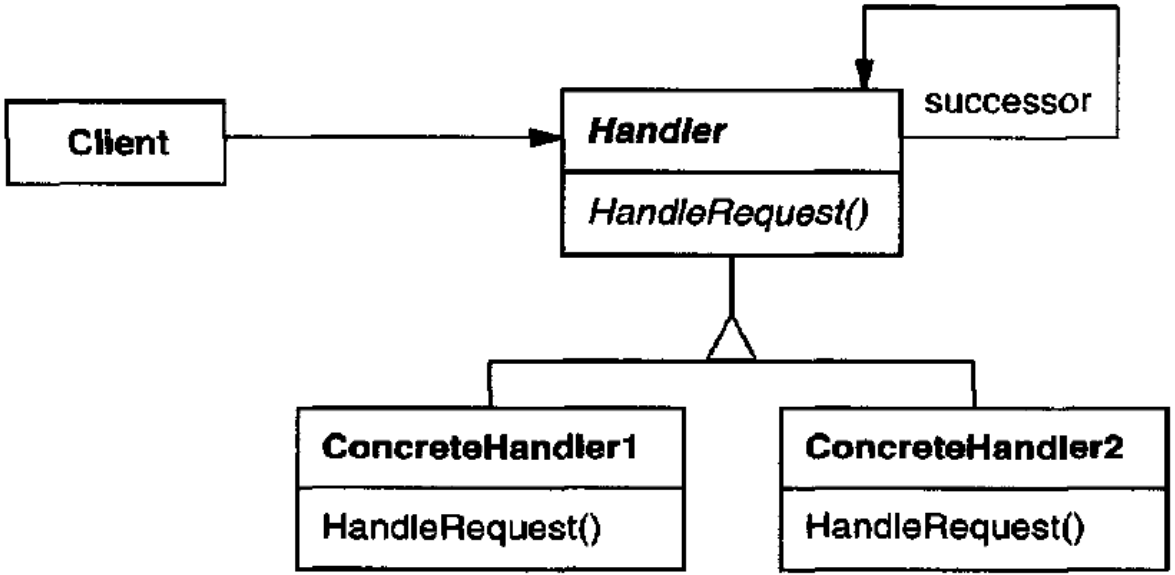
Chain of Responsibility

- It avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.
  - It chains the receiving objects and passes the request along the chain until an object handles it.

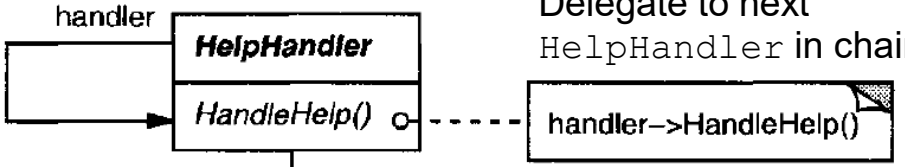
Java API example: The java awt/swing/JavaFx event handling mechanism.



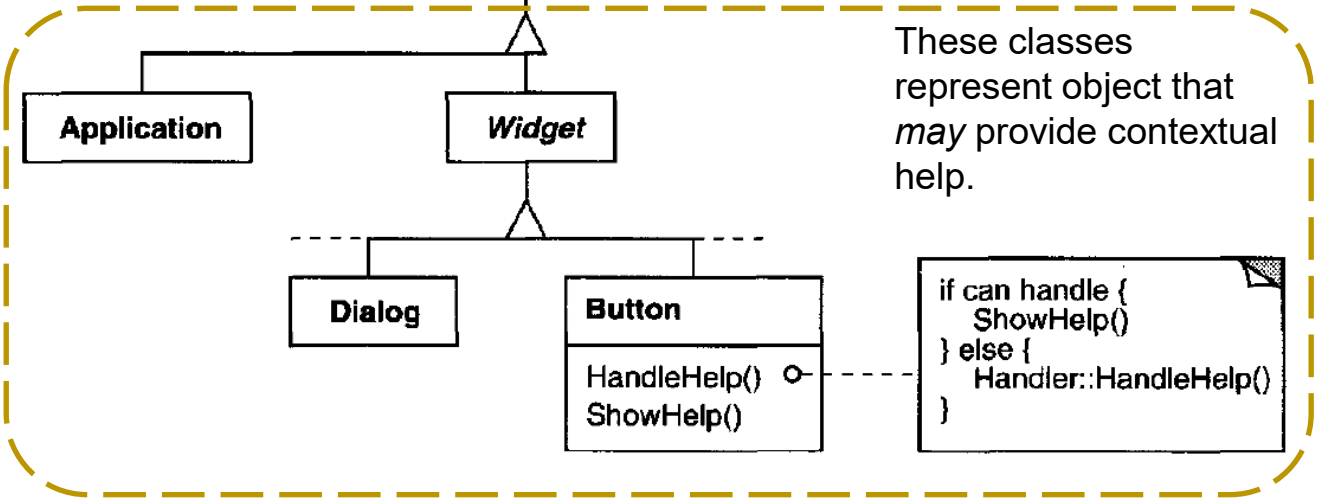
Chain of Responsibility



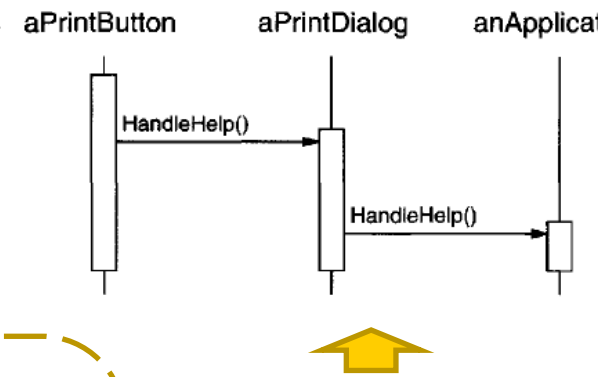
Chain of Responsibility



Delegate to next `HelpHandler` in chain.



These classes represent object that *may* provide contextual help.



Example where neither the print button nor the print dialog provide contextual help. Help is provided only at Application level.



Chain of Responsibility

Use when...

- ... more than one object may handle a request, and the handler isn't known a priori. The handler should be ascertained automatically.
- ... you want to issue a request to one of several objects without specifying the receiver explicitly.
- ... the set of objects that can handle a request should be specified dynamically.



Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. **Iterator**
5. Command
6. Memento
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter

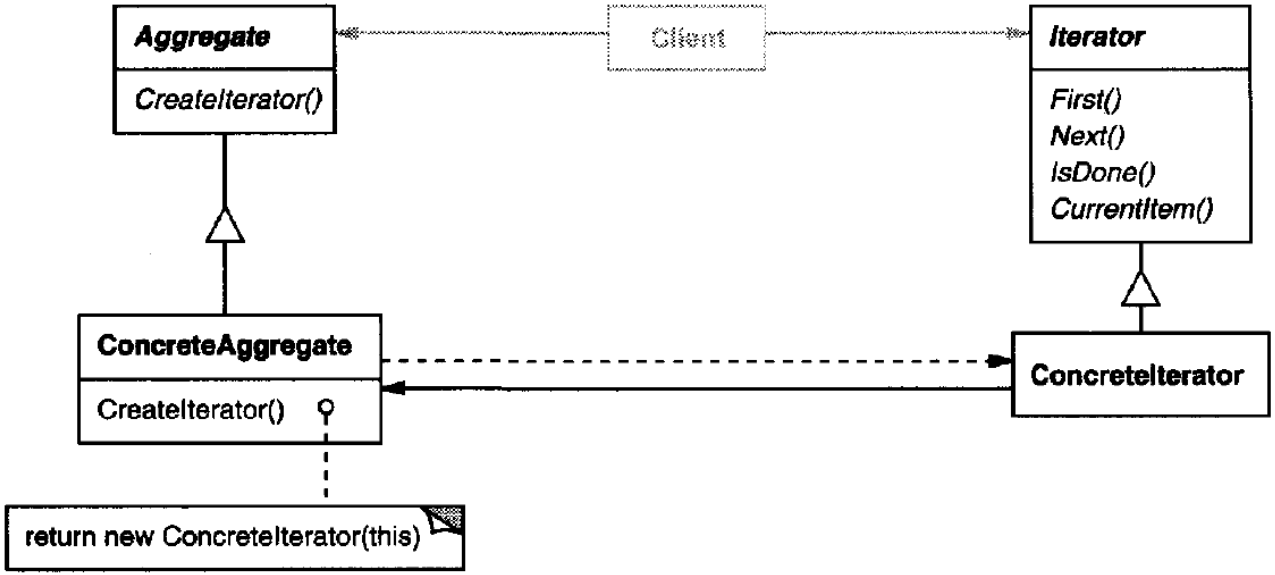


Iterator

- It provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
  - An aggregate object such as a list should give you a way to access its elements without exposing its internal structure.
  - Moreover, you might want to traverse the list in different ways, depending on what you want to accomplish.

Java API example: `java.util.Iterator`

Iterator


Aggregate
CreateIterator()
Client
Iterator
First()
Next()
IsDone()
CurrentItem()
ConcreteAggregate
CreateIterator()
ConcreteIterator
return new ConcreteIterator(this)



Iterator

Use to ...

- ... access an aggregate object's contents without exposing its internal representation.
- ... support multiple traversals of aggregate objects.
- ... to provide a uniform interface for traversing different aggregate structures (that is, to support polymorphic iteration).



Behavioral Patterns

1. Template Method
2. Observer
3. Chain of Responsibility
4. Iterator
5. **Command**
6. Memento
7. Strategy
8. State
9. Mediator
10. Visitor
11. Interpreter



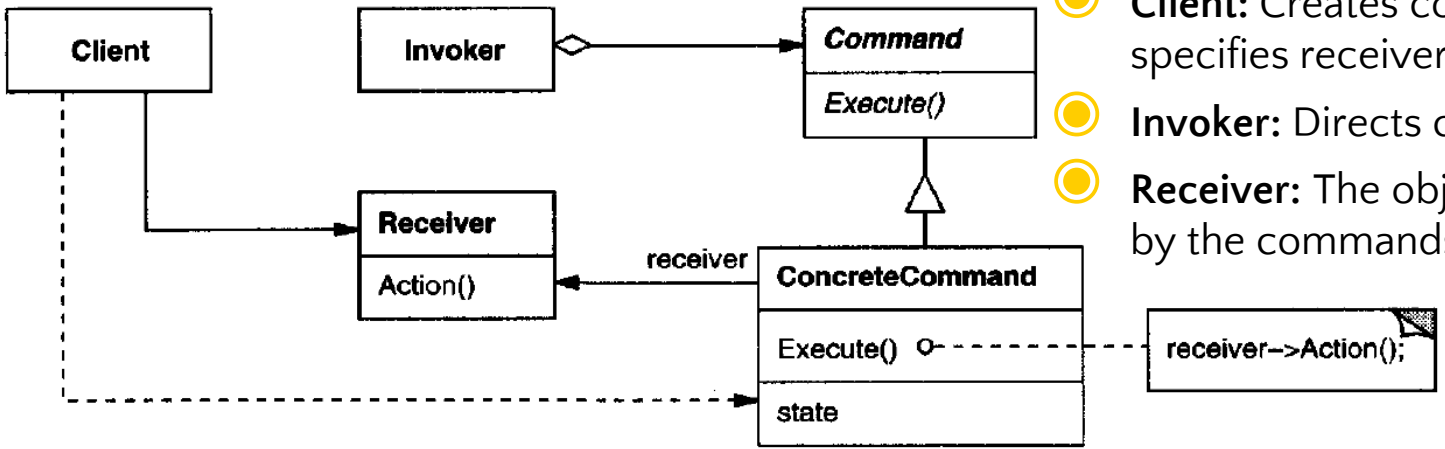
Command

- It encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
- Using this pattern, you encapsulate a method invocation process.

Java API example: the `java.lang.Runnable` interface

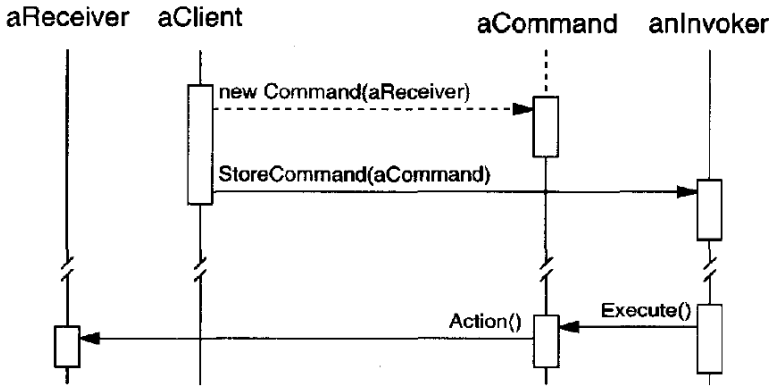


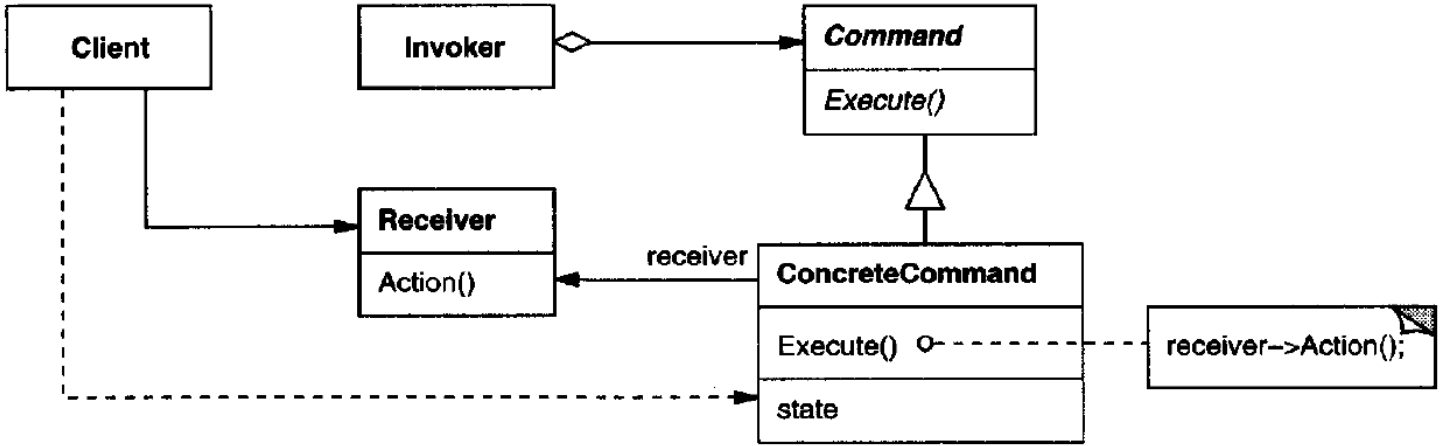
Command



- **Command:** Declares an interface for executing an operation
- **ConcreteCommand:** Implements command by applying appropriate actions on receiver.
- **Client:** Creates concrete command and specifies receiver.
- **Invoker:** Directs command to execute.
- **Receiver:** The object or model affected by the commands.

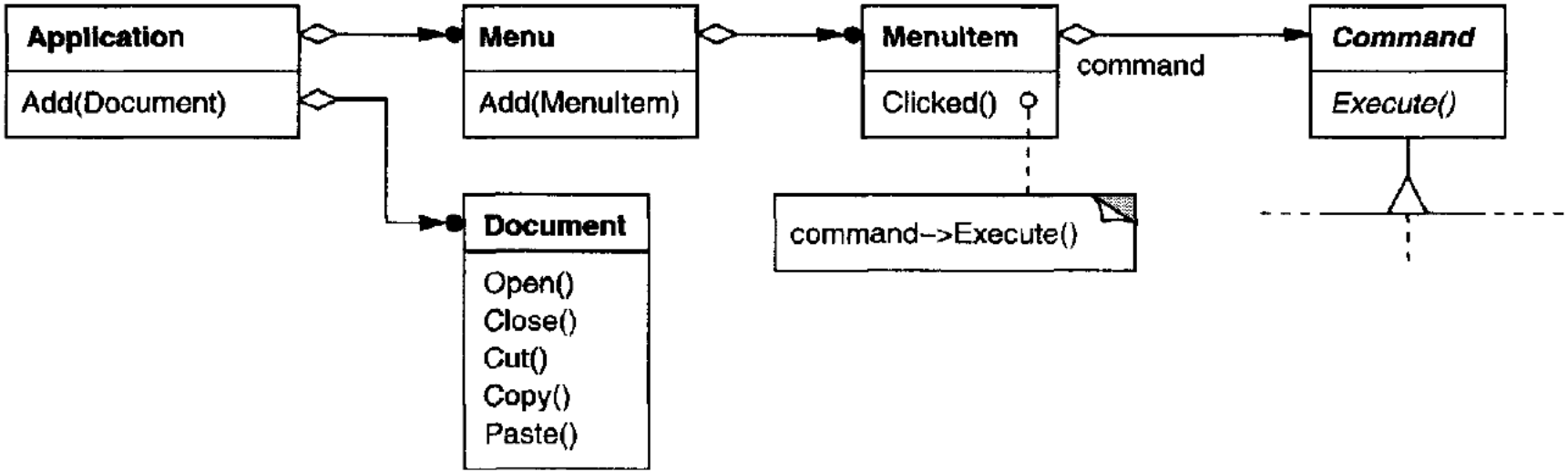
Command


aReceiver
aClient
aCommand
anInvoker
new Command(aReceiver)
StoreCommand(aCommand)
Execute()
Action()


Client
Invoker
Command
Execute()
Receiver
Action()
receiver
ConcreteCommand
Execute()
state
receiver->Action();

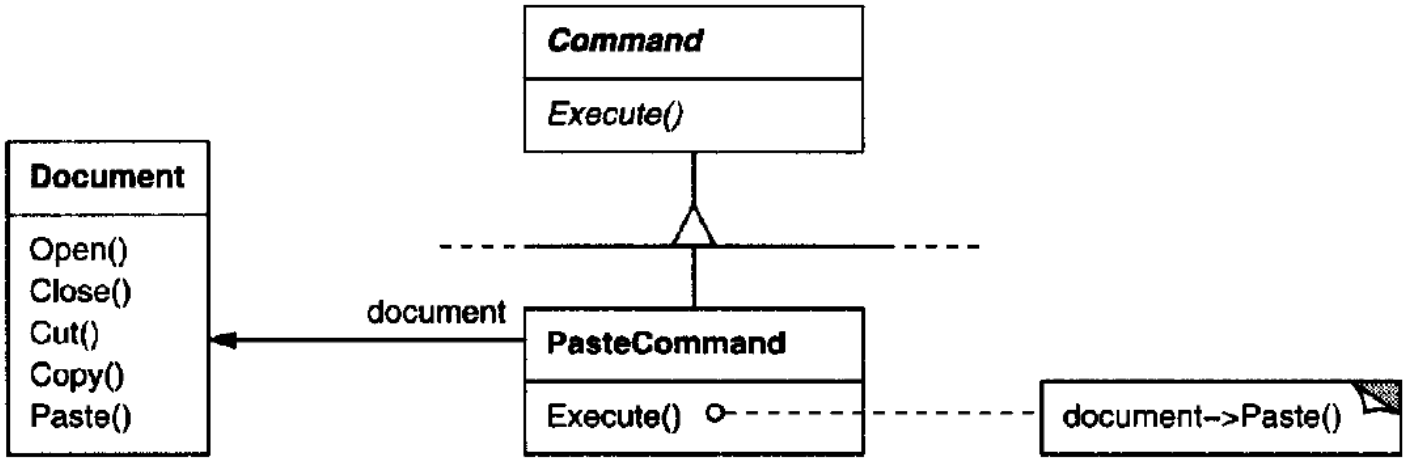
Command

- Using command in menu items



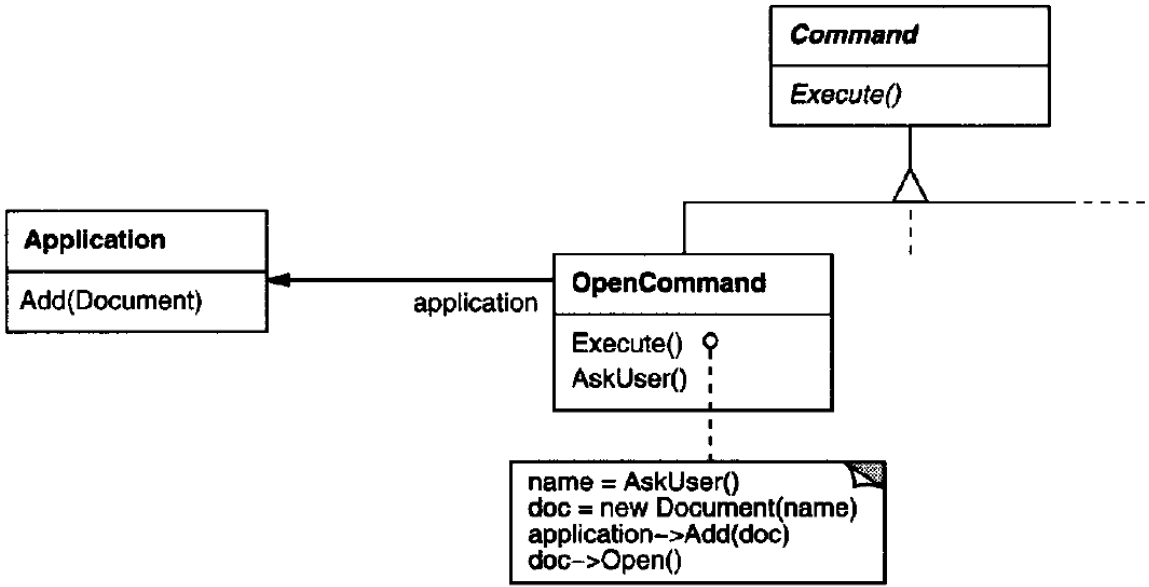
Command

- Using command for representing operations in document (1):



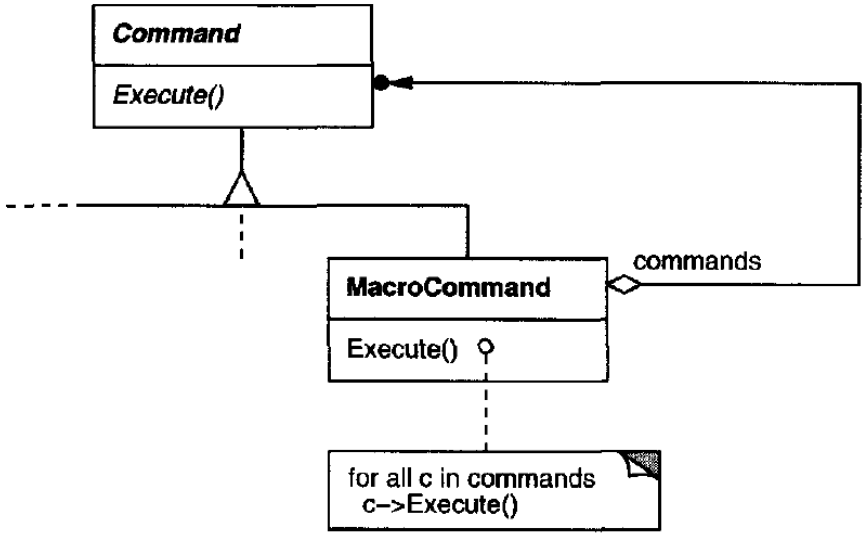
Command

- Using command for representing operations in document (2):



Command

- Using macro commands to aggregate multiple commands:





Command

- **Command** might also provide a method supporting the undoing of the command.
  - In this case, the undo method will take the necessary action to reverse its effects on the receiver.
- It decouples the object that invokes the operation from the one that knows how to perform it.
- **Commands** are first-class objects. They can be manipulated and extended like any other object.
- You can assemble commands into a composite command.
- It's easy to add new **Commands,** because you don't have to change existing classes